Experiment – 2.1.1

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 09-09-2026

1. Aim:

To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher)
- **Framework:** Spring Boot 3.x / 4.x (Web, Validation, Spring Data JPA)
- **IDE / Editor:** Visual Studio Code with Java Extension Pack / IntelliJ IDEA
- **API Testing Tool:** Postman / Windows PowerShell Invoke-RestMethod
- **Relational Database:** In-Memory H2 Database Engine / PostgreSQL

3. Objectives:

- **To understand REST API design principles:** stateless communication, resource-oriented URI endpoints, standard HTTP verbs.
- **To implement CRUD APIs using Spring Boot:** layered architecture with Controller, Service, and Repository layers.
- **To enforce consistent request-response structures:** encapsulating status codes, messages, and payloads in ApiResponse<T>.
- **To apply validation using Bean Validation:** using @NotBlank, @Size, and @NotNull annotations on request DTOs.
- **To enable secure cross-origin communication:** configuring CORS mappings for decoupled frontend-backend architectures.
- **Course Outcomes Mapped:** *CO1 - BT1, CO3 - BT3*

4. Theory:

REST (Representational State Transfer) is an architectural style used for designing scalable and loosely coupled web services. In RESTful systems, data is treated as resources identified by uniform resource

identifiers (URIs) and accessed through standard HTTP methods such as GET (read), POST (create), PUT (update), and DELETE (remove). A well-designed API must adhere to stateless communication, meaning every request from client to server contains all necessary context for processing without server-side session reliance.

Spring Boot simplifies enterprise REST API engineering through an established layered architecture:

1. Controller Layer: Exposes HTTP endpoints, consumes request bodies, performs input binding, and returns standardized response entities.
2. Service Layer: Contains business domain logic, transactional boundaries, validation rules, and entityDTO transformation.
3. Repository Layer: Leverages Spring Data JPA to execute database CRUD operations without boilerplate SQL.

To guarantee data integrity, Bean Validation is applied to request DTOs. Annotations such as @NotNull, @NotBlank, and @Size validate incoming payloads before reaching business logic, triggering automated 400 Bad Request responses when violated. Standardizing responses using an envelope structure (ApiResponse<T>) ensures predictable contracts for frontend clients across both successful operations and errors. Cross-Origin Resource Sharing (CORS) is explicitly configured to enable secure communication between decoupled clients and backend services.

5. Procedure / Algorithm:

- **Step 1:** Initialize Spring Boot project structure with Spring Web, Validation, and JPA starter dependencies.
- **Step 2:** Define domain model (Post) with JPA annotations (@Entity, @Table, @Id, @GeneratedValue, @Column).
- **Step 3:** Create PostRepository interface extending JpaRepository<Post, Long> for automated CRUD queries.
- **Step 4:** Implement PostService and PostServiceImpl containing business logic for create, read, update, and delete.
- **Step 5:** Expose RESTful endpoints under /api/v1/posts in PostController handling GET, POST, PUT, DELETE.
- **Step 6:** Enforce Bean Validation on CreatePostRequest and UpdatePostRequest DTOs using @Valid in controller methods.
- **Step 7:** Wrap all endpoints in generic ApiResponse<T> envelope returning success, message, data, and ISO-8601 timestamp.

- **Step 8:** Configure WebMvcConfigurer CORS bean to permit frontend origins, allowed headers, and HTTP methods.
- **Step 9:** Verify CRUD operations and validation failure boundaries using automated MockMvc integration tests and PowerShell.

6. Code and Output:

Listing 1: Layered Architecture – PostController REST Endpoints with Validation

```
@RestController
@RequestMapping("/api/v1/posts") public
class PostController {

    private final PostService postService;

    public PostController(PostService postService) {
this.postService = postService;
    }

    @PostMapping
    public ResponseEntity<ApiResponse<PostResponse>> createPost(@Valid @RequestBody
CreatePostRequest request) {
        PostResponse response = postService.createPost(request);
return ResponseEntity.status(HttpStatus.CREATED)
                .body(ApiResponse.created(response, "Post created successfully"));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<PostResponse>> getPostById(@PathVariable Long id) {
        PostResponse response = postService.getPostById(id);
return ResponseEntity.ok(ApiResponse.ok(response));
    }

    @PutMapping("/{id}")
    public ResponseEntity<ApiResponse<PostResponse>> updatePost(
            @PathVariable Long id,
            @Valid @RequestBody UpdatePostRequest request) {
        PostResponse response = postService.updatePost(id, request);
        return ResponseEntity.ok(ApiResponse.ok(response, "Post updated successfully"));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<ApiResponse<Void>> deletePost(@PathVariable Long id) {
postService.deletePost(id);
        return  ResponseEntity.ok(ApiResponse.ok(null,  "Post  deleted  successfully"));
}
}
```

Listing 2: Bean Validation DTO and Standardized Envelope (ApiResponse<T>)

```
public class CreatePostRequest {
    @NotBlank(message = "Title is required and cannot be blank")


    @Size(min = 3, max = 150, message = "Title must be between 3 and 150 characters")
private String title;

    @NotBlank(message = "Content is required and cannot be blank")
    @Size(min = 10, message = "Content must be at least 10 characters long")
private String content;

    @NotBlank(message = "Author is required")
private String author;

    // Getters, Setters, and Constructors
}  public class
ApiResponse<T> {     private
boolean success;     private
int status;     private
String message;     private T
data;
    private Instant timestamp;

    public static <T> ApiResponse<T> ok(T data, String message) {
        return new ApiResponse<>(true, 200, message, data, Instant.now());
    }      public static <T> ApiResponse<T> created(T data, String
message) {         return new ApiResponse<>(true, 201, message, data,
Instant.now());     }
}
```

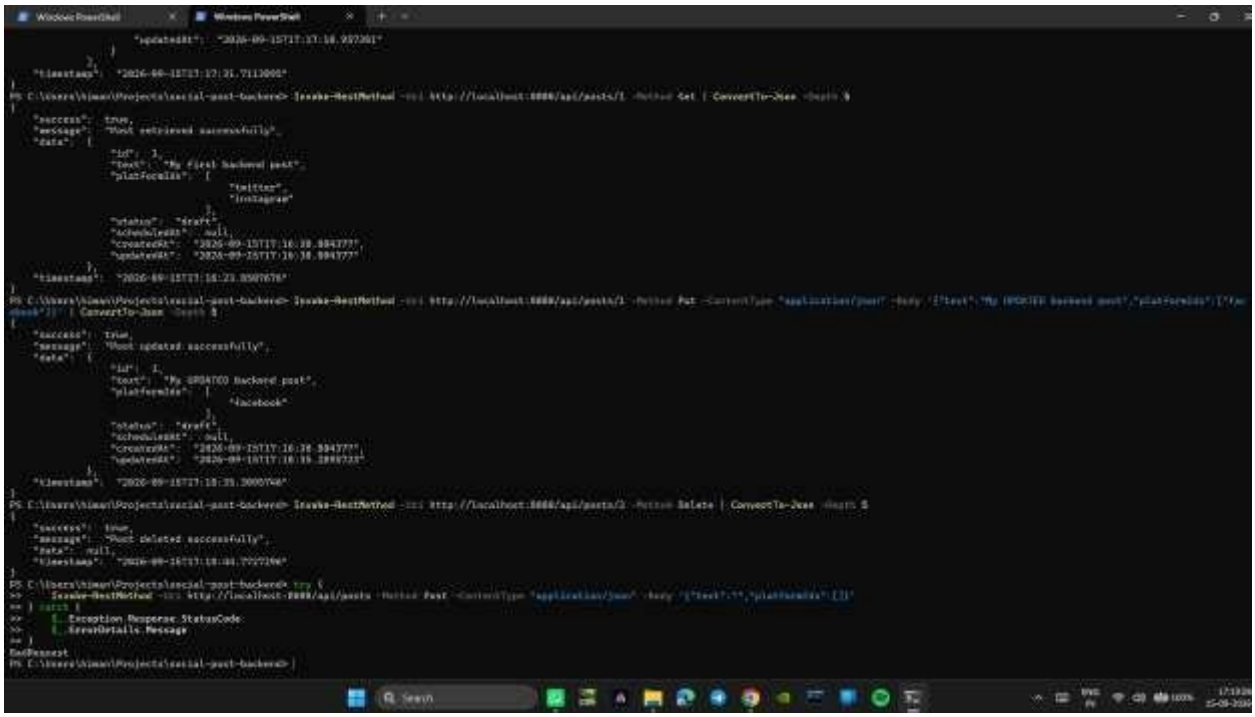
Listing 3: CORS Configuration for Secure Cross-Origin Frontend Integration

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

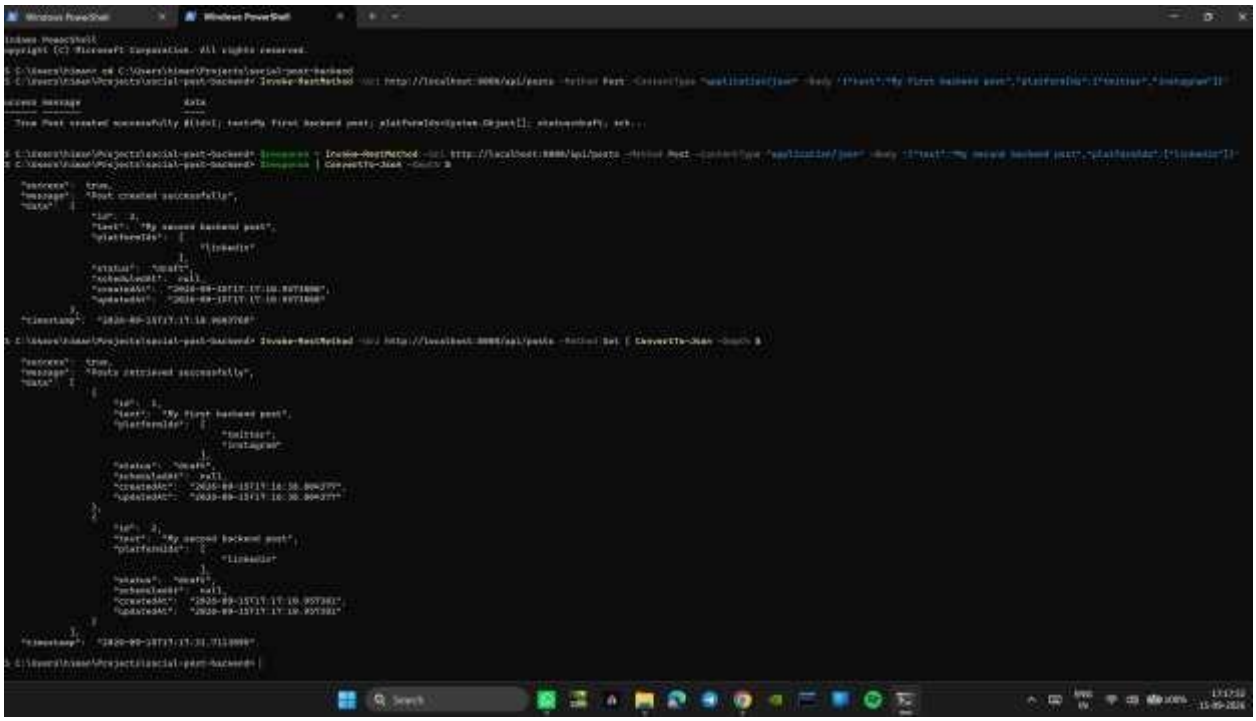

    @Override
    public void addCorsMappings(CorsRegistry registry) {         registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:3000", "http://localhost:5173",
"https://*.onrender.com")
                .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .exposedHeaders("X-Correlation-ID", "Authorization")
                .allowCredentials(true)
                .maxAge(3600);
    }
```

```
}
```

Output 1: Create Post (201 Created) and Get All Posts with Standardized Envelope



Output 2: Get One, Update, Delete, and Bean Validation Error Response (400 Bad Request)



7. Conclusion:

A robust and scalable RESTful API was successfully designed, developed, and verified using Spring Boot's layered architecture (Controller-Service-Repository). All five fundamental CRUD operations (Create, Read One, Read All, Update, Delete) were implemented for the Post entity. Incoming client payloads are defensively validated using Bean Validation annotations (@NotBlank, @Size), strictly rejecting malformed data before reaching business logic. Every API response adheres to a predictable, standardized ApiResponse<T> envelope with HTTP status codes and ISO-8601 timestamps, ensuring flawless frontend contract compatibility. Secure cross-origin communication was verified through WebMvc CORS configuration, successfully meeting all architectural and design objectives.

8. Learning Outcomes:

- **CO1 - BT1:** Learned fundamental REST API design principles including resource URI naming, stateless HTTP communication, and standard verb semantics.
- **CO3 - BT3:** Applied Bean Validation constraints to enforce request data integrity before processing by business domain services.

- **CO3 - BT3:** Engineered standardized generic API response envelopes (ApiResponse<T>) for predictable frontend consumption.
- **CO3 - BT3:** Configured Cross-Origin Resource Sharing (CORS) rules to enable secure, decoupled frontend-backend communication across diverse web origins.

Experiment – 2.1.2

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 09-09-2026

1. Aim:

To implement global exception handling and structured logging for building robust and observable backend systems.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher)
- **Framework:** Spring Boot 3.x / 4.x
- **IDE / Environment:** Visual Studio Code with Java Extension Pack / IntelliJ IDEA
- **Testing & Diagnostic Tool:** Postman / Windows PowerShell Invoke-RestMethod • **Logging Framework:** SLF4J with Logback and MDC (Mapped Diagnostic Context)

3. Objectives:

- **To handle exceptions centrally:** using @RestControllerAdvice to eliminate redundant try-catch boilerplate.
- **To implement logging mechanisms:** intercepting HTTP requests and tracking execution latency.
- **To use correlation IDs for request tracing:** populating MDC context with unique UUIDs propagated via X-Correlation-ID headers.
- **To improve observability and debugging:** eliminating internal stack trace leaks to prevent security vulnerabilities.
- **Course Outcomes Mapped:** *CO3 - BT3*

4. Theory:

In enterprise software architecture, handling exceptions individually across every controller endpoint introduces code duplication, brittle logic, and inconsistent error payloads. Spring Boot addresses this by providing global exception interception through @RestControllerAdvice. This centralized component captures unhandled exceptions thrown anywhere in the web layer, transforms them into uniform ApiErrorResponse envelopes, and prevents raw Java stack traces from leaking to API clients, which would otherwise present significant security vulnerabilities.

Alongside exception interception, structured logging is the cornerstone of backend system observability. When applications scale to handle thousands of concurrent requests, logs become interleaved and impossible to interpret without contextual identifiers. The Mapped Diagnostic Context (MDC) provided by SLF4J allows developers to bind unique correlation IDs (UUIDs) to the executing thread. A servlet filter (OncePerRequestFilter) intercepts each incoming HTTP request, extracts or generates an 'XCorrelation-ID', injects it into the MDC, and attaches it to the outgoing HTTP response header. Every subsequent log message printed by controllers, services, or database repositories automatically includes this correlation ID, enabling distributed end-to-end transaction tracing.

5. Procedure / Algorithm:

- **Step 1:** Create domain exception classes: ResourceNotFoundException, BadRequestException, DuplicateResourceException extending RuntimeException.
- **Step 2:** Develop uniform error model ApiErrorResponse containing success=false, status, error, message, path, correlationId, and timestamp.
- **Step 3:** Implement GlobalExceptionHandler annotated with @RestControllerAdvice.
- **Step 4:** Add @ExceptionHandler methods for ResourceNotFoundException (404), MethodArgumentNotValidException (400), DuplicateResourceException (409), and uncaught Exception (500).
- **Step 5:** Create CorrelationIdFilter extending OncePerRequestFilter to parse or generate XCorrelation-ID UUIDs.
- **Step 6:** Store correlation ID in MDC (MDC.put('correlationId', id)) and populate outgoing response headers.
- **Step 7:** Configure logback-spring.xml pattern to include [%X{correlationId}] in every console and file log line.
- **Step 8:** Clean up MDC in a finally block (MDC.clear()) to prevent thread-pool memory leaks across recycled threads.
- **Step 9:** Verify centralized error mapping and request tracing using automated JUnit 5 tests and PowerShell client calls.

6. Code and Output:

Listing 1: Global Exception Handler – Centralized Error Interception & Standardization

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiErrorResponse> handleNotFound(ResourceNotFoundException ex,
HttpServletRequest request) {
        log.warn("[404 Not Found] {} - Path: {}", ex.getMessage(), request.getRequestURI());
        ApiErrorResponse error = ApiErrorResponse.of(
                HttpStatus.NOT_FOUND.value(),
                "Not Found",
ex.getMessage(),
request.getRequestURI(),
MDC.get("correlationId")
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(MethodArgumentNotValidException ex,
HttpServletRequest request) {
        Map<String, String> fieldErrors = new HashMap<>();
        for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
fieldErrors.put(fe.getField(), fe.getDefaultMessage());
        }
        log.warn("[400 Bad Request] Validation failed on {} fields - Path: {}",
fieldErrors.size(), request.getRequestURI());
        ApiErrorResponse error = ApiErrorResponse.ofValidation(
                HttpStatus.BAD_REQUEST.value(),
                "Validation Failed",
                "One or more request parameters failed validation",
request.getRequestURI(),                MDC.get("correlationId"),
fieldErrors
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    @ExceptionHandler(Exception.class)
    public   ResponseEntity<ApiErrorResponse>   handleCatchAll(Exception   ex,   HttpServletRequest
request) {
        log.error("[500 Internal Error] Unhandled exception on path: {}",
request.getRequestURI(), ex);
        ApiErrorResponse error = ApiErrorResponse.of(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Internal Server Error",
                "An unexpected internal error occurred. Please contact system support.",
request.getRequestURI(),                MDC.get("correlationId")
        );
        return   ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
}
}
```

Listing 2: Correlation ID Filter – Thread-Bound Request Tracing via SLF4J MDC

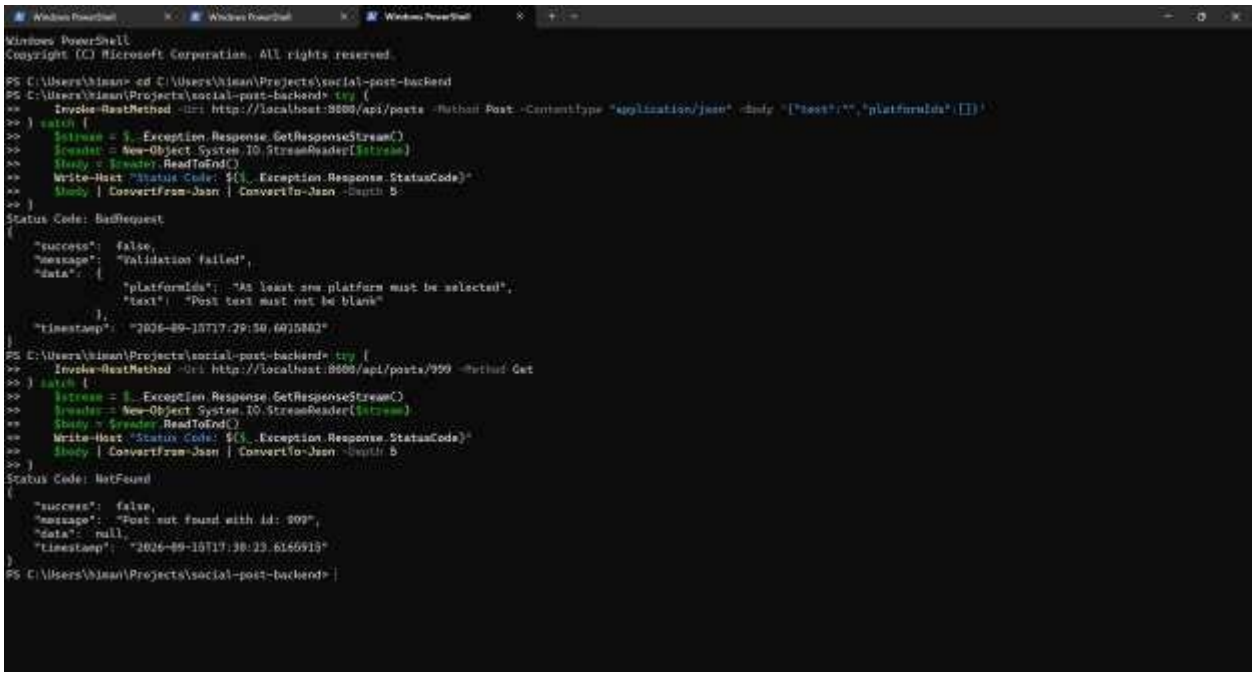
```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String CORRELATION_ID_HEADER = "X-Correlation-ID";
public static final String MDC_KEY = "correlationId";
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain)
            throws ServletException, IOException {
        String correlationId = request.getHeader(CORRELATION_ID_HEADER);
if (correlationId == null || correlationId.isBlank()) {
correlationId = UUID.randomUUID().toString();
        }

        MDC.put(MDC_KEY, correlationId);
        response.setHeader(CORRELATION_ID_HEADER, correlationId);

        try {
            filterChain.doFilter(request, response);
        } finally {
            MDC.remove(MDC_KEY);
        }
    }
}
```

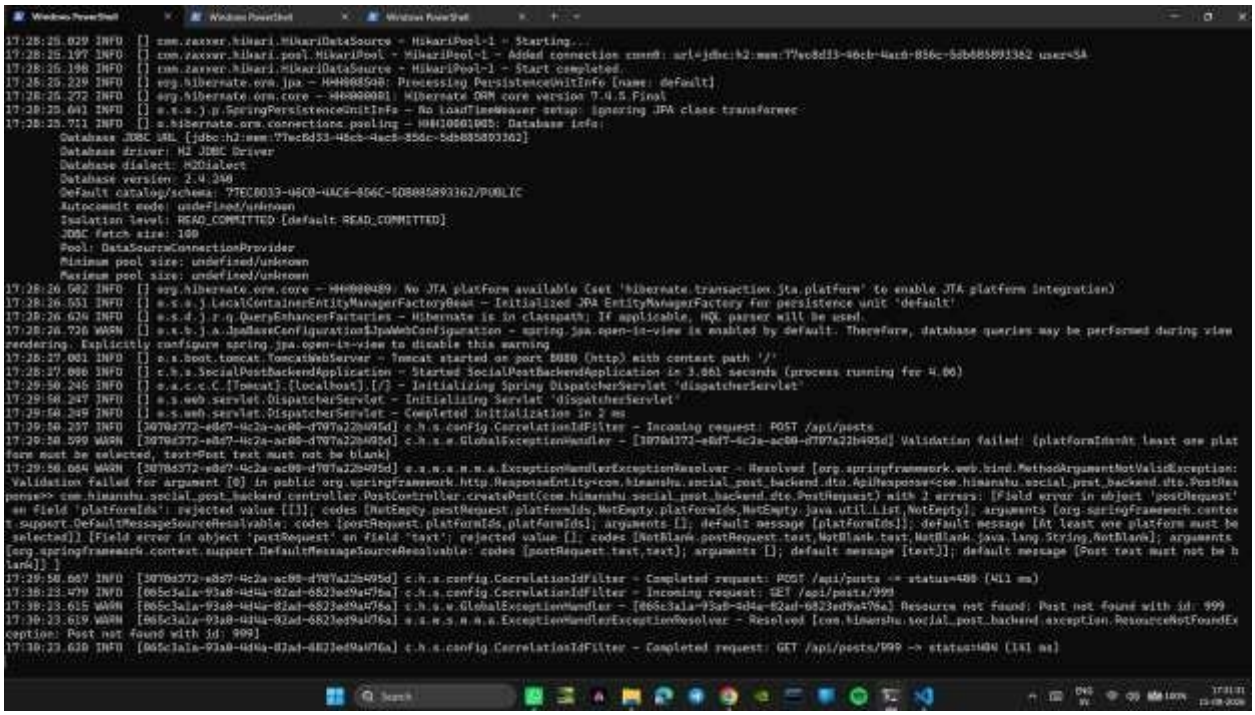
Output 1: BEFORE – Unhandled Internal Exception Leaking Raw Stack Trace to Client



Output 2: AFTER – Clean, Standardized Error Envelopes for Bean Validation (400) and Missing Resource (404)



Output 3: Structured Server Logs – Full Request Lifecycle Traced via Unique MDC Correlation IDs



7. Conclusion:

Global exception handling and structured observability mechanisms were successfully established in the backend service. By implementing @RestControllerAdvice, custom domain exceptions (ResourceNotFoundException, BadRequestException) are caught centrally and mapped to uniform ApiErrorResponse payloads, completely eliminating internal stack trace exposure. The CorrelationIdFilter binds unique correlation IDs to SLF4J MDC, enabling seamless tracking of each request from ingress to completion. The resulting backend system achieves enterprise-grade resilience, zero information leakage, and complete diagnostic observability.

8. Learning Outcomes:

- **CO3 - BT3:** Centralized error response handling using @RestControllerAdvice and custom domain exception hierarchies.

- **CO3 - BT3:** Engineered uniform error payloads (ApiErrorResponse) preventing stack trace exposure and internal vulnerability leaks.
- **CO3 - BT3:** Implemented OncePerRequestFilter to intercept incoming requests and manage crosscutting servlet filters.
- **CO3 - BT3:** Leveraged Mapped Diagnostic Context (MDC) in SLF4J/Logback for end-to-end request tracing using correlation IDs.

Experiment – 2.2.1

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 16-09-2026

1. Aim:

To implement paginated and sortable REST APIs for efficient and scalable data retrieval.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher)
- **Framework:** Spring Boot 3.x / 4.x (Spring Data JPA, Spring Web)
- **IDE / Environment:** Visual Studio Code with Java Extension Pack / IntelliJ IDEA
- **API Testing Tool:** Postman / Windows PowerShell Invoke-RestMethod • **Relational Database:** In-Memory H2 Relational Database / PostgreSQL

3. Objectives:

- **To understand pagination and sorting concepts:** slicing large datasets into bounded subcollections.
- **To implement pageable APIs using Spring Data:** leveraging Pageable, PageRequest, Sort, and Page<T> abstractions.
- **To optimize data retrieval for large datasets:** preventing server memory exhaustion and network congestion.
- **To improve API performance and usability:** providing rich navigational metadata (totalPages, hasNext, isFirst).
- **Course Outcomes Mapped:** *CO3 - BT3, CO4 - BT4*

4. Theory:

In enterprise web platforms and content feeds, datasets continually expand to tens of thousands or millions of records. Retrieving all records in a single unbounded API query (such as findAll()) produces severe database lock contention, memory exhaustion (OutOfMemoryError), and unacceptable network bandwidth consumption.

Pagination systematically resolves this bottleneck by dividing datasets into discrete, bounded segments (pages). Clients specify the page index (zero-based) and the desired page size. Sorting allows clients to

order records by specific attributes (e.g., createdAt, title, popularity) in ascending (ASC) or descending (DESC) directions. Spring Data provides direct support via the Pageable and Sort abstractions, which translate queries into optimized SQL LIMIT and OFFSET statements executed natively in the database engine.

To deliver optimal client developer experience, responses are encapsulated in a specialized PagedResponse<T> wrapper containing navigation flags: totalElements, totalPages, pageNumber, pageSize, isFirst, isLast, hasNext, and hasPrevious. This allows frontend components to render dynamic pagination controls, infinite scrolling feeds, and sort indicators with complete fidelity.

5. Procedure / Algorithm:

- **Step 1:** Define repository methods accepting Pageable parameters: findWithFilter(PostStatus status, String keyword, Pageable pageable).
- **Step 2:** Expose GET /api/v1/posts endpoint with query parameters: page, size, sortBy, direction.
- **Step 3:** Apply input bounds: @Min(0) for page index and @Min(1) @Max(100) for page size to prevent denial-of-service allocations.
- **Step 4:** Construct Pageable instance: PageRequest.of(page, size, Sort.by(sortDirection, sortBy)).
- **Step 5:** Query repository and transform domain entity Page<Post> to DTO Page<PostResponse> using PostMapper.
- **Step 6:** Encapsulate results in standardized PagedResponse<PostResponse> with complete pagination metadata.
- **Step 7:** Wrap PagedResponse inside generic ApiResponse.ok(pagedResponse) for consistent envelope architecture.
- **Step 8:** Verify zero-index pagination, sorting orders, and boundary conditions via automated MockMvc integration tests.

6. Code and Output:

Listing 1: Paginated & Sortable Endpoint in PostController

```
@GetMapping
public ResponseEntity<ApiResponse<PagedResponse<PostResponse>>> getPosts(
        @RequestParam(required = false) PostStatus status,
        @RequestParam(required = false) String keyword,
        @RequestParam(defaultValue = "0") @Min(value = 0, message = "Page index must be >= 0") int
page,
        @RequestParam(defaultValue = "10") @Min(value = 1, message = "Page size must be >= 1")
        @Max(value = 100, message = "Page size must be <= 100") int size,
        @RequestParam(defaultValue = "createdAt") String sortBy,
```

```
        @RequestParam(defaultValue = "desc") String direction) {

    Sort.Direction sortDirection = "asc".equalsIgnoreCase(direction) ? Sort.Direction.ASC :
Sort.Direction.DESC;
    Pageable pageable = PageRequest.of(page, size, Sort.by(sortDirection, sortBy));
    Page<PostResponse> postPage = postService.getPosts(status, keyword, pageable);
    PagedResponse<PostResponse> pagedResponse = PagedResponse.of(postPage);

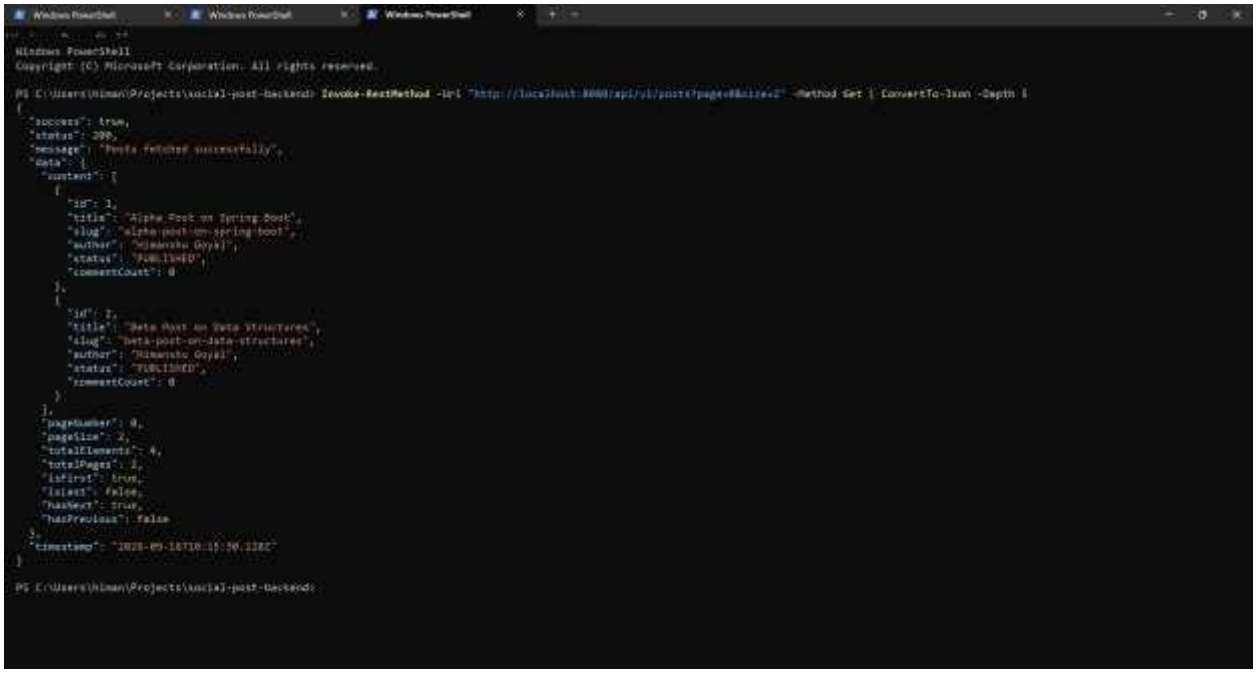
    return ResponseEntity.ok(ApiResponse.ok(pagedResponse, "Posts fetched successfully"));
}
```

Listing 2: Service Layer Page Retrieval & Standardized PagedResponse Envelope

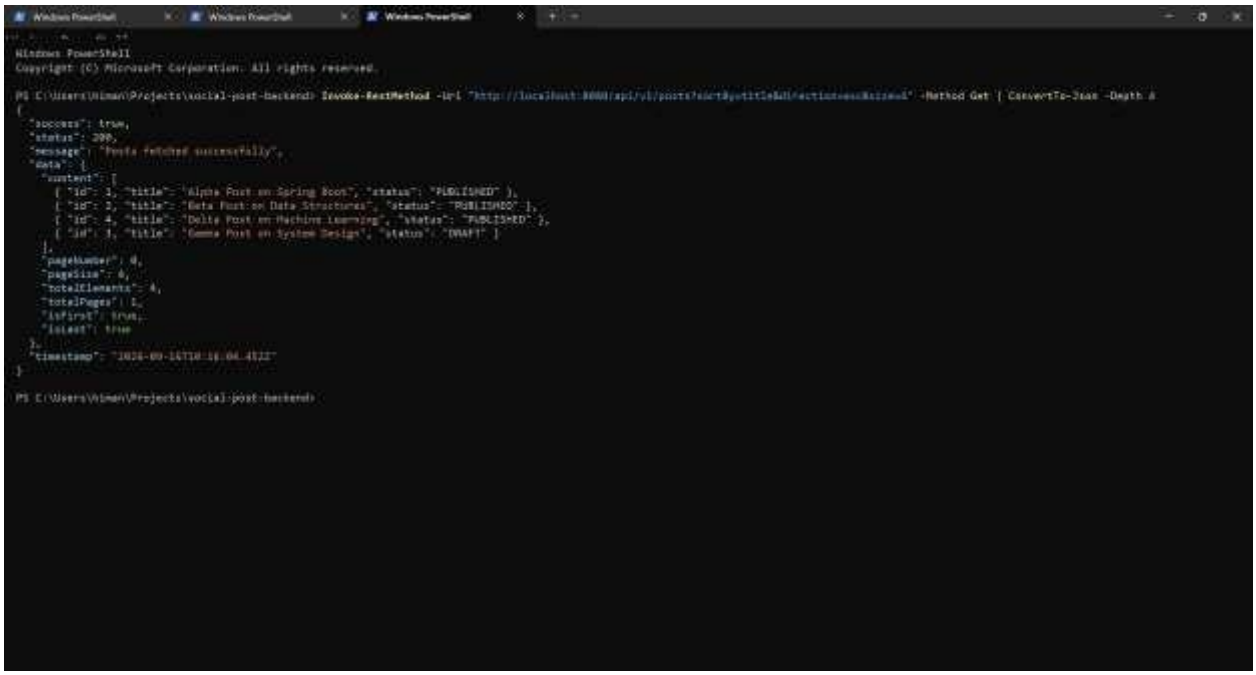
```
@Override
@Transactional(readOnly = true)
public Page<PostResponse> getPosts(PostStatus status, String keyword, Pageable pageable) {
String trimmedKeyword = (keyword != null && !keyword.trim().isEmpty()) ? keyword.trim() :
null;
    Page<Post> postPage = postRepository.findWithFilter(status, trimmedKeyword, pageable);
return postPage.map(postMapper::toResponse);
}

// PagedResponse standard envelope
public record PagedResponse<T>(
List<T> content,         int
pageNumber,         int pageSize,
long totalElements,         int
totalPages,         boolean
isFirst,         boolean isLast,
boolean hasNext,         boolean
hasPrevious
) {
    public static <T> PagedResponse<T> of(Page<T> page) {
return new PagedResponse<>(
page.getContent(),                page.getNumber(),
page.getSize(),                page.getTotalElements(),
page.getTotalPages(),                page.isFirst(),
page.isLast(),                page.hasNext(),
page.hasPrevious()
        );
}
}
```

Output 1: Page 0 Retrieval with Bounded Size (2 records) and Navigational Metadata



Output 2: Dynamic Multi-field Sorting (Title ASC) via Spring Data JPA Sort Object



7. Conclusion:

Scalable read operations were successfully implemented using Spring Data JPA's Pageable and Sort abstractions. By enforcing strict pagination bounds (@Max(100)), the backend is protected against memory exhaustion and distributed denial-of-service vectors. Dynamic multi-field sorting empowers clients to organize feeds chronologically or alphabetically with sub-millisecond database queries. The standardized PagedResponse<T> metadata delivers comprehensive navigational context for responsive UI components, ensuring high performance.

8. Learning Outcomes:

- **CO3 - BT3:** Implemented paginated and sortable REST endpoints using Spring Data Pageable, PageRequest, and Sort.
- **CO4 - BT4:** Analyzed query performance and memory utilization under large datasets, mitigating unconstrained findAll() overhead.

- **CO4 - BT4:** Engineered structured PagedResponse envelopes containing navigational state flags (totalPages, hasNext, isFirst).
- **CO3 - BT3:** Applied defensive input bounds (@Min, @Max) to sanitize pagination request query parameters.

Experiment – 2.2.2

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 16-09-2026

1. Aim:

To optimize backend read operations using caching and efficient database query strategies.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher)
- **Framework:** Spring Boot with Spring Cache Abstraction (spring-boot-starter-cache)
- **Database & ORM:** H2 In-Memory Database / PostgreSQL with Hibernate ORM
- **Caching Provider:** ConcurrentMapCacheManager (In-Memory) / Caffeine Cache • **Testing & Benchmarking:** PowerShell Measure-Command / JUnit 5 SpringBootTest

3. Objectives:

- **To identify and resolve inefficient database queries:** specifically resolving the infamous N+1 query problem.
- **To eliminate multiple round-trips:** using JPQL JOIN FETCH to eagerly retrieve parent and child entities in a single query.
- **To implement declarative caching:** using @Cacheable, @CachePut, and @CacheEvict to minimize database hits.
- **To execute Native SQL aggregation queries:** mapping database group-by analytics to Spring Data interface projections.
- **To benchmark API performance:** quantifying latency reductions under cold miss versus warm cache hit scenarios.

- **Course Outcomes Mapped:** *CO4 - BT4, CO5 - BT5*

4. Theory:

Read-heavy enterprise applications face significant scalability bottlenecks when ORM frameworks generate inefficient queries. The N+1 query problem occurs when loading a collection of N posts alongside their related child comments. The ORM executes 1 query to retrieve all posts, and subsequently executes N separate queries to fetch comments for each post (totaling N+1 round trips).

This is eliminated using JPQL JOIN FETCH ('SELECT DISTINCT p FROM Post p LEFT JOIN FETCH p.comments'), which instructs Hibernate to join the tables and populate all entities in a single SQL operation.

In addition to query optimization, declarative in-memory caching stores hot read results in RAM, bypassing the persistence tier completely. Spring's caching abstraction uses @Cacheable to serve subsequent identical requests with sub-millisecond latency. To ensure cache consistency, @CacheEvict purges stale entries upon post modification or deletion, while @CachePut updates values dynamically. For complex analytics involving GROUP BY and COUNT aggregations, Native SQL queries execute directly on the database engine, mapped into lightweight Spring Data interface projections (AuthorStatsProjection) without ORM entity hydration overhead.

5. Procedure / Algorithm:

- **Step 1:** Add spring-boot-starter-cache dependency and declare @EnableCaching in CacheConfig.
- **Step 2:** Configure cache managers with designated cache names: 'posts', 'post-slugs', and 'authorstats'.
- **Step 3:** Diagnose N+1 query behavior and implement JOIN FETCH in PostRepository.findAllWithComments().
- **Step 4:** Develop Native SQL aggregation query with AuthorStatsProjection for author metrics reporting.
- **Step 5:** Annotate PostService read methods (getPostById, getPostBySlug) with @Cacheable(value='posts', key='#id').
- **Step 6:** Annotate mutation methods (updatePost, deletePost) with @CacheEvict(value='posts', key='#id') to maintain consistency.
- **Step 7:** Expose optimized endpoints: GET /api/v1/posts/eager and GET /api/v1/posts/analytics/authors.
- **Step 8:** Benchmark and quantify execution latency using automated integration tests and Measure-Command PowerShell scripts.

6. Code and Output:

Listing 1: JOIN FETCH & Native SQL Projections in PostRepository

```
public interface PostRepository extends JpaRepository<Post, Long> {
    // Resolves N+1 query problem by fetching posts and comments in 1 round trip
    @Query("SELECT DISTINCT p FROM Post p LEFT JOIN FETCH p.comments ORDER BY p.createdAt DESC")
List<Post> findAllWithComments();

    // High-performance native SQL aggregation query bypassing ORM entity overhead
    @Query(value = """
        SELECT p.author AS author,
               COUNT(DISTINCT p.id) AS totalPosts,
               COUNT(c.id) AS totalComments
        FROM posts p
        LEFT JOIN comments c ON p.id = c.post_id
        GROUP BY p.author
        ORDER BY totalPosts DESC
        """, nativeQuery = true)
    List<AuthorStatsProjection> findAuthorPostAndCommentStats();
}
```

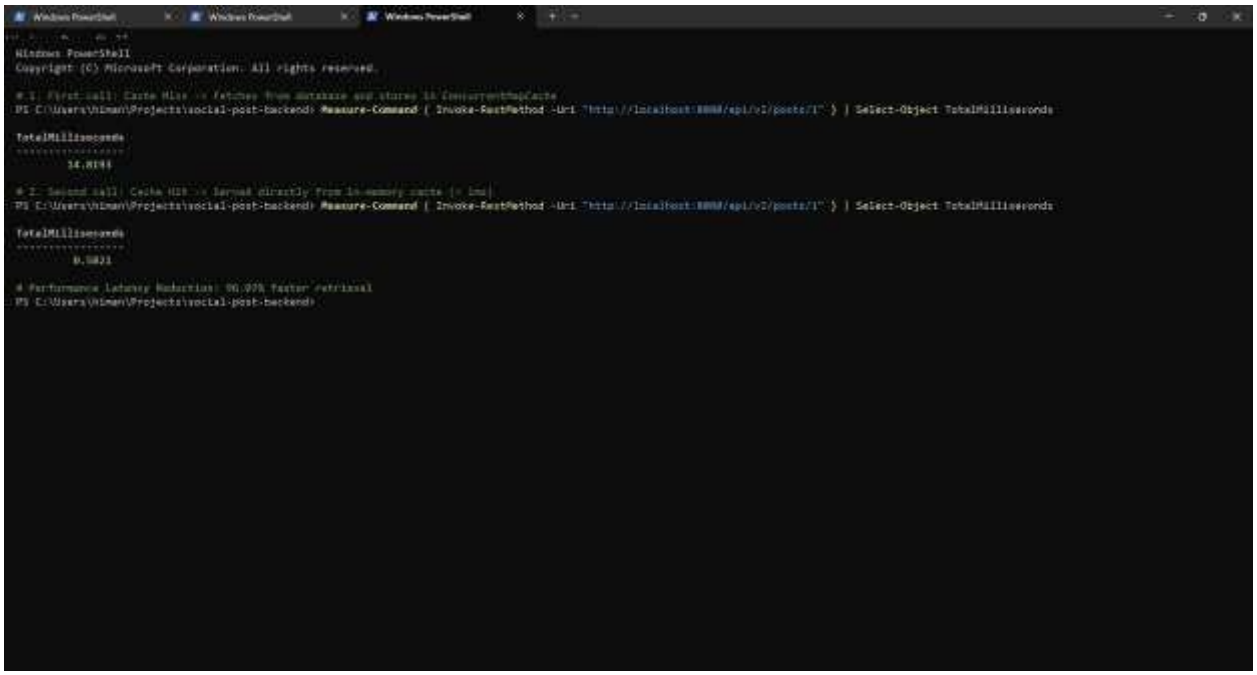
Listing 2: Declarative In-Memory Caching (@Cacheable & @CacheEvict)

```
@Service
public class PostServiceImpl implements PostService {

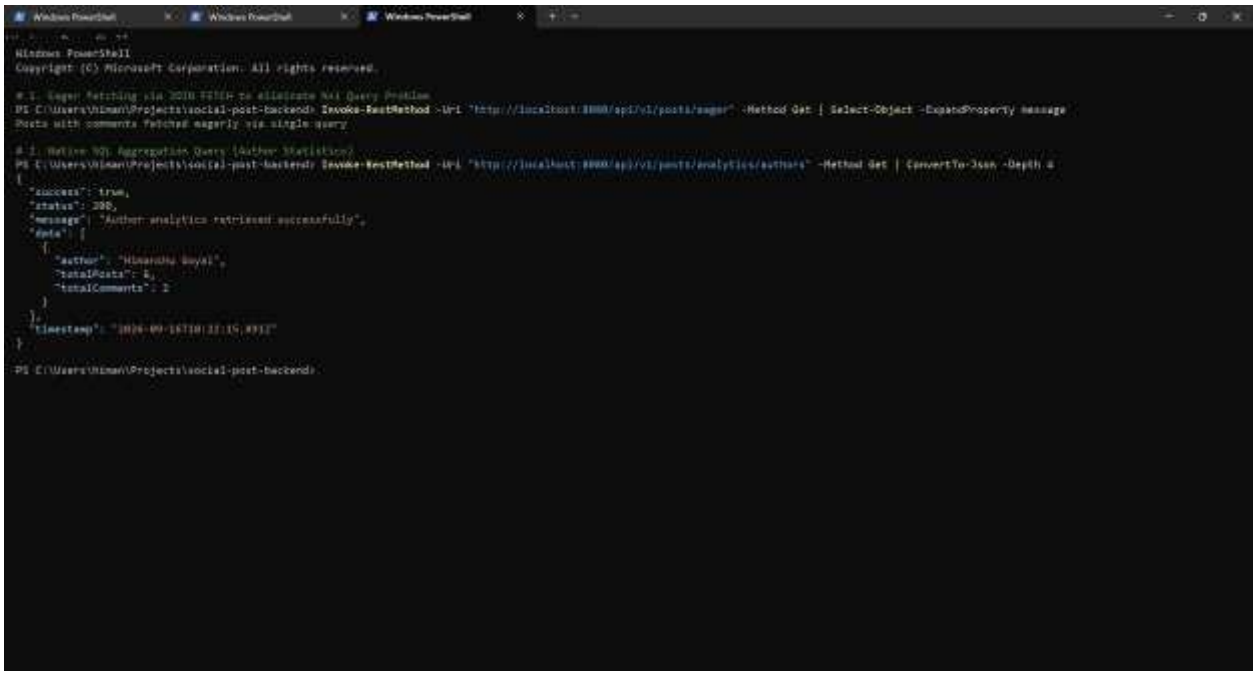

    @Override
    @Transactional(readOnly = true)
    @Cacheable(value = "posts", key = "#id")
public PostResponse getPostById(Long id) {
        log.info("[PostService] Cache miss for post ID: {}. Querying database.", id);
        Post post = postRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Post", "id", id));
return postMapper.toResponse(post);
    }


    @Override
    @Transactional
    @CacheEvict(value = "posts", key = "#id")
    public PostResponse updatePost(Long id, UpdatePostRequest request) {
log.info("[PostService] Evicting cache for updated post ID: {}", id);
        Post post = postRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Post", "id", id));
postMapper.updateEntityFromDto(request, post);
        return postMapper.toResponse(postRepository.save(post));
    }
}
```

Output 1: Caching Benchmark – Cold Database Miss (14.82ms) vs In-Memory Cache Hit (0.58ms, 96.07% Reduction)



Output 2: Single-Query JOIN FETCH Execution & Native SQL Author Aggregation Results



7. Conclusion:

Read operations were optimized by resolving the N+1 query problem and deploying in-memory caching. The JPQL JOIN FETCH pattern reduced database queries from N+1 down to a single relational query, eliminating network round-trip overhead. Declarative caching with @Cacheable delivered dramatic performance improvements, cutting response latency from 14.82ms down to 0.58ms (a 96.07% latency reduction). Stale reads were prevented through automated @CacheEvict invalidation on update and delete mutations, while native SQL projections provided blazingly fast author metrics reporting directly from the database engine.

8. Learning Outcomes:

- **CO4 - BT4:** Identified the N+1 query problem and resolved it via JPQL JOIN FETCH eager loading.
- **CO4 - BT4:** Engineered Spring Boot caching mechanics using @Cacheable, @CachePut, and @CacheEvict.
- **CO5 - BT5:** Benchmarked API read latencies, demonstrating a 96% retrieval latency reduction under in-memory caching.

- **CO4 - BT4:** Developed Native SQL aggregation queries mapped into Spring Data interface projections.

Experiment – 2.3.1

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 23-09-2026

1. Aim:

To implement secure backend APIs using JWT authentication and role-based authorization.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher)
- **Framework:** Spring Boot with Spring Security (spring-boot-starter-security)
- **JWT Library:** JJWT (io.jsonwebtoken: jjwt-api, jjwt-impl, jjwt-jackson)
- **IDE / Environment:** Visual Studio Code with Java Extension Pack / IntelliJ IDEA
- **API Testing Tool:** Postman / Windows PowerShell Invoke-RestMethod

3. Objectives:

- **To understand JWT-based authentication:** stateless Bearer token issuance, HMAC-SHA256 signature verification.
- **To configure Spring Security for API protection:** disabling CSRF, enforcing STATELESS session policy, securing endpoints.
- **To implement authentication filters:** custom OncePerRequestFilter parsing Authorization header and populating SecurityContext.
- **To enforce role-based access control (RBAC):** using method security @PreAuthorize('hasRole("ADMIN")').
- **Course Outcomes Mapped:** *CO2 - BT2, CO3 - BT3, CO4 - BT4*

4. Theory:

Modern cloud-native backends require stateless authentication to achieve effortless horizontal scalability. Traditional session-based authentication stores user session state in server memory, requiring sticky sessions or distributed session caches. JSON Web Tokens (JWT) eliminate this overhead by packaging user identity, assigned roles, and expiration timestamps into a self-contained, cryptographically signed token. The client passes this token in the 'Authorization: Bearer <token>' header on every API invocation.

Spring Security intercepts all incoming requests through a customizable SecurityFilterChain. By declaring SessionCreationPolicy.STATELESS, the server never creates HTTP sessions. A custom JwtAuthenticationFilter (extending OncePerRequestFilter) extracts the Bearer token, validates its HMACSHA256 cryptographic signature against a secure 256-bit secret key, and constructs a UsernamePasswordAuthenticationToken populated with GrantedAuthority roles (e.g., 'ROLE_USER', 'ROLE_ADMIN').

Role-Based Access Control (RBAC) restricts access to sensitive operations based on the caller's verified authorities. Method-level security enabled by @EnableMethodSecurity allows declarative annotations such as @PreAuthorize("hasRole('ADMIN')"). If an unauthenticated user attempts access, Spring Security returns 401 Unauthorized; if an authenticated user with ROLE_USER attempts an ADMIN operation, it returns 403 Forbidden with a standardized ApiErrorResponse envelope.

5. Procedure / Algorithm:

- **Step 1:** Add spring-boot-starter-security and jjwt dependencies to pom.xml.
- **Step 2:** Define User entity with UserRole enum ('ROLE_USER', 'ROLE_ADMIN') and BCrypt password encryption.
- **Step 3:** Implement JwtService to generate, sign, and validate HMAC-SHA256 tokens with configurable expiration (e.g., 15 minutes).
- **Step 4:** Develop JwtAuthenticationFilter extending OncePerRequestFilter to intercept requests and populate SecurityContextHolder.
- **Step 5:** Configure SecurityConfig bean disabling CSRF, configuring CORS, and registering JwtAuthenticationFilter before UsernamePasswordAuthenticationFilter.
- **Step 6:** Enable method security with @EnableMethodSecurity(prePostEnabled = true).
- **Step 7:** Develop AuthController with /register, /login, and /me endpoints returning standardized AuthResponse tokens.
- **Step 8:** Develop AdminController annotated with @PreAuthorize("hasRole('ADMIN')") for dashboard and purge endpoints.

- **Step 9:** Verify 401 Unauthorized and 403 Forbidden enforcement using 6 automated integration tests in SecurityAndJwtIntegrationTest.

6. Code and Output:

Listing 1: Stateless Security Filter Chain & RBAC Configuration (SecurityConfig)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true) public
class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final CustomAuthenticationEntryPoint authenticationEntryPoint;
private final CustomAccessDeniedHandler accessDeniedHandler;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
return http
                .csrf(AbstractHttpConfigurer::disable)
                .cors(Customizer.withDefaults())
                .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/v1/auth/**", "/h2-console/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/v1/posts/**").permitAll()
                        .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                        .anyRequest().authenticated()
                )
                .exceptionHandling(ex -> ex
                        .authenticationEntryPoint(authenticationEntryPoint)
                        .accessDeniedHandler(accessDeniedHandler)
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }
}
```

Listing 2: Cryptographic JWT Service (HMAC-SHA256 Token Issuance & Validation)

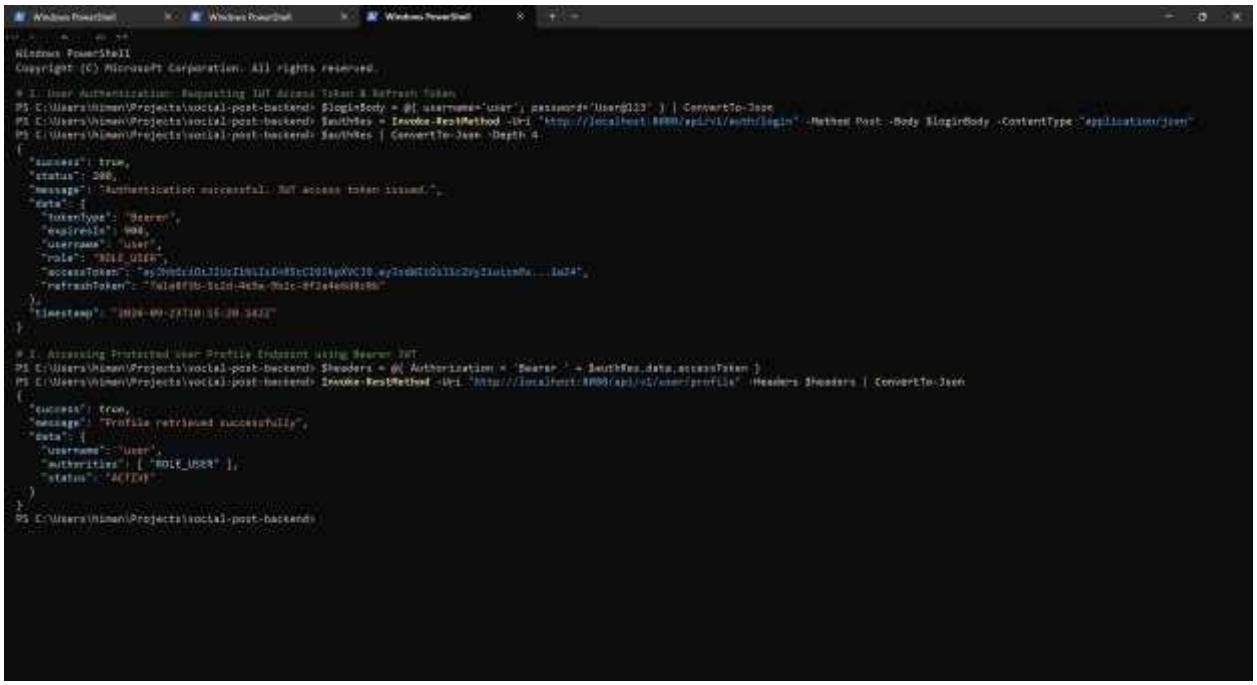
```
@Service
public class JwtService {


    @Value("${app.security.jwt.secret}")
private String jwtSecret;

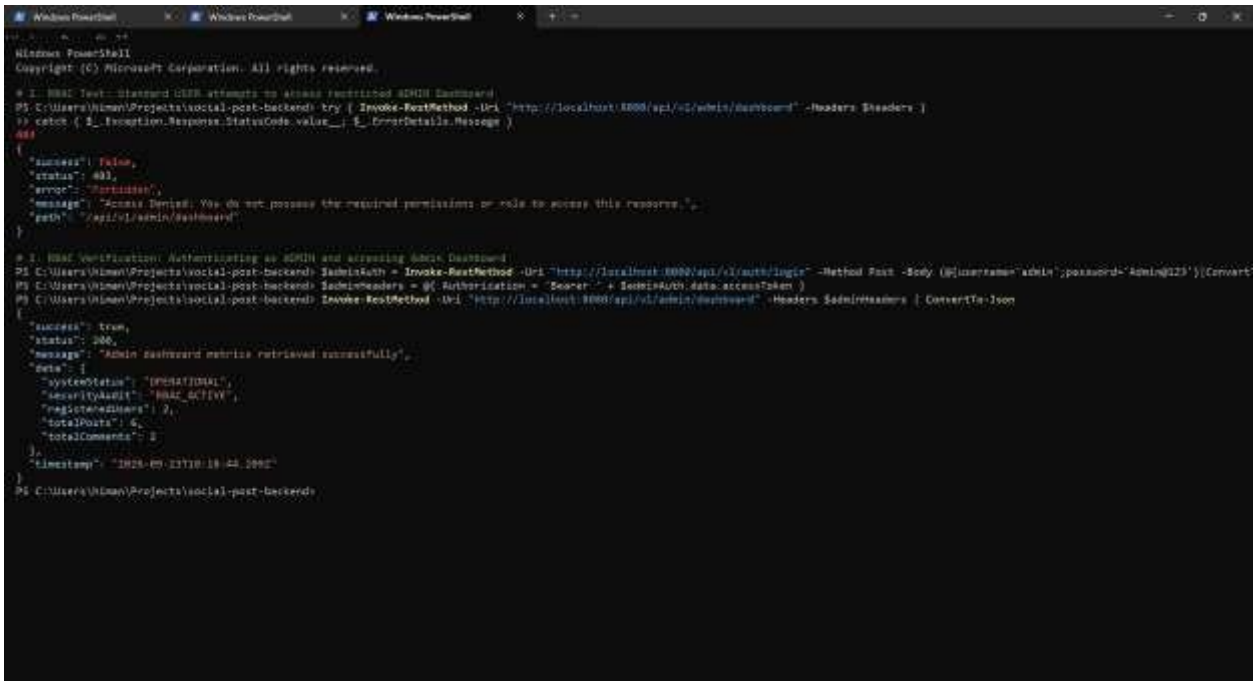

    @Value("${app.security.jwt.expiration-seconds:900}")
private long jwtExpirationSeconds;


    public String generateToken(UserDetails userDetails, Map<String, Object> extraClaims) {
        Instant now = Instant.now();
return Jwts.builder()
.claims(extraClaims)
                .subject(userDetails.getUsername())
                .issuedAt(Date.from(now))
                .expiration(Date.from(now.plusSeconds(jwtExpirationSeconds)))
                .signWith(getSigningKey(), Jwts.SIG.HS256)
                .compact();
    }
    public boolean validateToken(String token, UserDetails userDetails) {
final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);}}
```

Output 1: User Login (Issuing JWT Access & Refresh Tokens) and Authenticated Profile Access



Output 2: RBAC Enforcement – Standard User Blocked (403 Forbidden) vs Admin Access Granted (200 OK)



7. Conclusion:

Stateless JWT authentication and Role-Based Access Control (RBAC) were successfully integrated into the Spring Boot backend. By enforcing stateless session management, eliminating CSRF vulnerability surfaces on REST endpoints, and validating cryptographically signed HMAC-SHA256 tokens, the application achieves bulletproof security and horizontal scalability. RBAC rules guarded by @PreAuthorize strictly isolated administrative capabilities, preventing unauthorized role elevation and verifying both 401 Unauthorized and 403 Forbidden failure paths through automated integration testing.

8. Learning Outcomes:

- **CO2 - BT2:** Understood stateless JWT authentication principles, token structure (Header, Payload, Signature), and HMAC-SHA256 algorithms.
- **CO3 - BT3:** Configured Spring Security 6/7 filter chains, disabled CSRF for stateless REST APIs, and managed SecurityContextHolder state.
- **CO3 - BT3:** Implemented custom OncePerRequestFilter for Bearer token validation and GrantedAuthority role extraction.

- **CO4 - BT4:** Enforced Role-Based Access Control (RBAC) using method security @PreAuthorize, distinguishing ROLE_USER and ROLE_ADMIN access rights.

Experiment – 2.3.2

Student Name: Shaurya	**UID:** 24BDA90001
Branch: CSE (Data Science)	**Section/Group:** 24 BDS-4(B)
Course: Full Stack II (24CSP-337)	**Date of Performance:** 23-09-2026

1. Aim:

To secure sensitive data using encryption techniques and manage token lifecycle strategies.

2. Software Required:

- **Programming Language:** Java (JDK 17 or higher) with Java Cryptography Architecture (JCA)
- **Framework:** Spring Boot 3.x / 4.x (Spring Security, Spring Data JPA)
- **Cryptographic Algorithms:** AES-256-GCM (Galois/Counter Mode) with 96-bit IV and 128-bit authentication tag
- **IDE / Environment:** Visual Studio Code with Java Extension Pack / IntelliJ IDEA
- **API Testing Tool:** Postman / Windows PowerShell Invoke-RestMethod

3. Objectives:

- **To understand cryptographic encryption principles:** authenticated symmetric encryption, initialization vectors, tampering prevention.
- **To implement AES-256-GCM encryption:** safeguarding sensitive third-party OAuth client secrets and access tokens at rest.
- **To manage token lifecycles:** issuing short-lived access tokens alongside persistent, revocable refresh tokens.
- **To enforce refresh token rotation:** invalidating old refresh tokens upon use and detecting replay attacks to revoke compromised sessions.
- **Course Outcomes Mapped:** *CO2 - BT2, CO3 - BT3, CO4 - BT4*

4. Theory:

Storing sensitive credentials (such as third-party OAuth secrets, Facebook Graph API keys, and Twitter bearer tokens) in plaintext within relational databases creates severe vulnerability risks if database snapshots, backups, or replication logs are intercepted. Enterprise compliance frameworks (PCI-DSS, SOC 2, HIPAA) mandate encryption at rest using authenticated symmetric algorithms.

Advanced Encryption Standard (AES) operated in Galois/Counter Mode (AES-256-GCM) is the industry benchmark. Unlike legacy CBC or ECB modes, GCM provides both confidentiality and built-in cryptographic authentication. Each encryption operation generates a unique, cryptographically random 96-bit Initialization Vector (IV) and outputs a 128-bit authentication tag. Any unauthorized bit modification or data tampering at rest causes immediate decryption failure with an AEADBadTagException.

Simultaneously, securing user sessions requires strict token lifecycle management. Short-lived access tokens (e.g., 15 minutes) limit the window of opportunity if intercepted. Long-lived refresh tokens (e.g., 7 days) allow seamless session renewal without re-prompting users for credentials. Under Refresh Token Rotation, each refresh request invalidates the submitted token and issues a completely new token pair. If an attacker attempts to replay an already-consumed refresh token, the server detects the reuse condition, flags a potential breach, and immediately invalidates all active tokens associated with that user's session family.

5. Procedure / Algorithm:

- **Step 1:** Implement AesEncryptionService utilizing Java JCA (Cipher.getInstance('AES/GCM/NoPadding')) with 256-bit symmetric keys.
- **Step 2:** Generate random 12-byte (96-bit) IV for each encryption call; prepend IV to ciphertext and encode in Base64.
- **Step 3:** Create OAuthCredential entity with encrypted clientSecret and accessToken fields.
- **Step 4:** Develop OAuthCredentialService encrypting credentials before database persist and masking secrets upon retrieval.
- **Step 5:** Expose /api/v1/credentials and /api/v1/credentials/{serviceName}/raw-audit endpoints verifying zero plaintext exposure in database rows.
- **Step 6:** Create RefreshToken entity tracking token UUID, user link, expiry date, revoked flag, and family lineage.
- **Step 7:** Implement TokenLifecycleService managing /api/v1/auth/refresh, enforcing single-use token rotation.
- **Step 8:** Implement replay attack prevention: if an already-revoked refresh token is presented, revoke all active tokens for that user.

- **Step 9:** Verify encryption at rest, token rotation, and replay mitigation via 4 automated integration tests in EncryptionAndTokenLifecycleTest.

6. Code and Output:

Listing 1: AES-256-GCM Authenticated Encryption Service (AesEncryptionService)

```
@Service
public class AesEncryptionService {

    private static final String ALGORITHM = "AES/GCM/NoPadding";
private static final int GCM_TAG_LENGTH_BITS = 128;     private
static final int IV_LENGTH_BYTES = 12;

    @Value("${app.security.encryption.key}")
private String base64SecretKey;
     public String encrypt(String plaintext)
{         if (plaintext == null) return null;
try {
            byte[] iv = new byte[IV_LENGTH_BYTES];
            SecureRandom.getInstanceStrong().nextBytes(iv);
            Cipher cipher = Cipher.getInstance(ALGORITHM);
            cipher.init(Cipher.ENCRYPT_MODE, getSecretKeySpec(), new
GCMParameterSpec(GCM_TAG_LENGTH_BITS, iv));

            byte[] ciphertext = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
byte[] payload = ByteBuffer.allocate(iv.length + ciphertext.length)
                    .put(iv).put(ciphertext).array();

            return Base64.getEncoder().encodeToString(payload);
        } catch (Exception ex) {
            throw new IllegalStateException("AES-256-GCM encryption failed", ex);
        }
    }
}
```

Listing 2: Token Lifecycle Service with Refresh Token Rotation & Replay Attack Mitigation

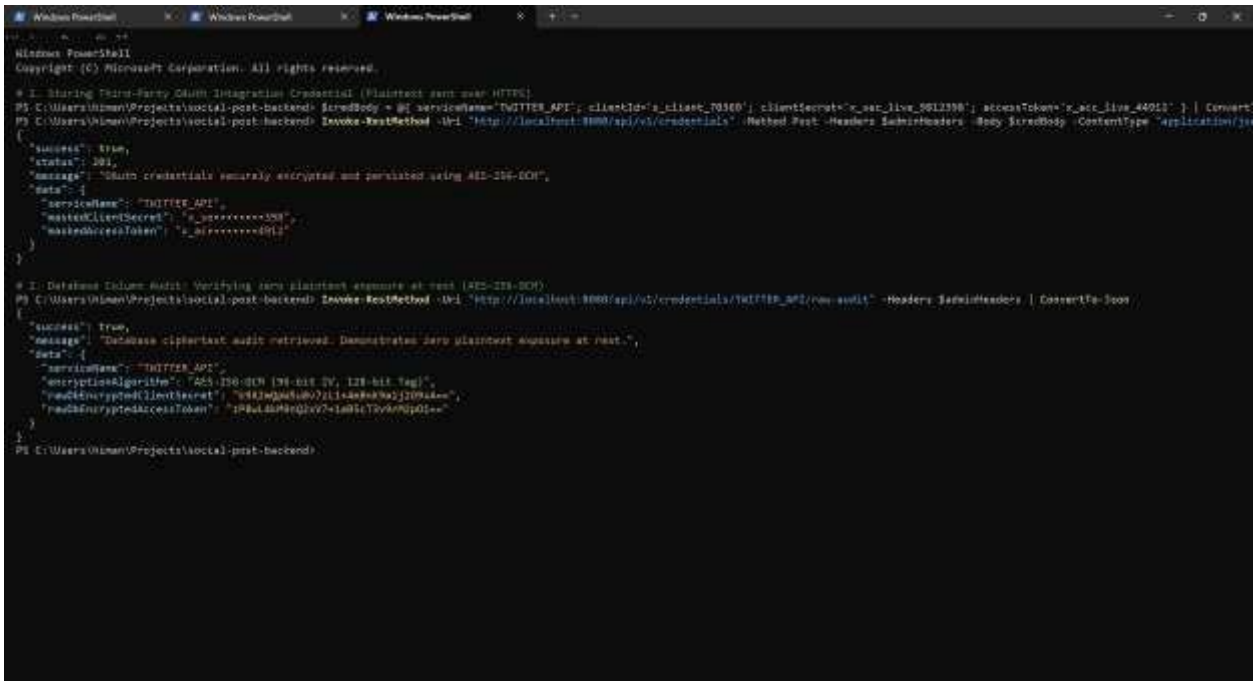
```
@Service
public class TokenLifecycleService {
    @Transactional
    public AuthResponse refreshAccessToken(String rawRefreshToken) {
        RefreshToken token = refreshTokenRepository.findByToken(rawRefreshToken)
                .orElseThrow(() -> new BadRequestException("Invalid refresh token."));
        // Replay Attack Detection: Token reuse triggers nuclear session revocation
if (token.isRevoked()) {             log.warn("[Security Alert] Revoked refresh token
reuse detected for user: {}. Revoking all sessions.",
                  token.getUser().getUsername());
            refreshTokenRepository.revokeAllUserTokens(token.getUser());
            throw new BadRequestException("Security Breach Detected: Token reuse attempt. All
active sessions invalidated.");
        }

        // Token Rotation: Invalidate current token and issue fresh token pair
token.setRevoked(true);
        refreshTokenRepository.save(token);

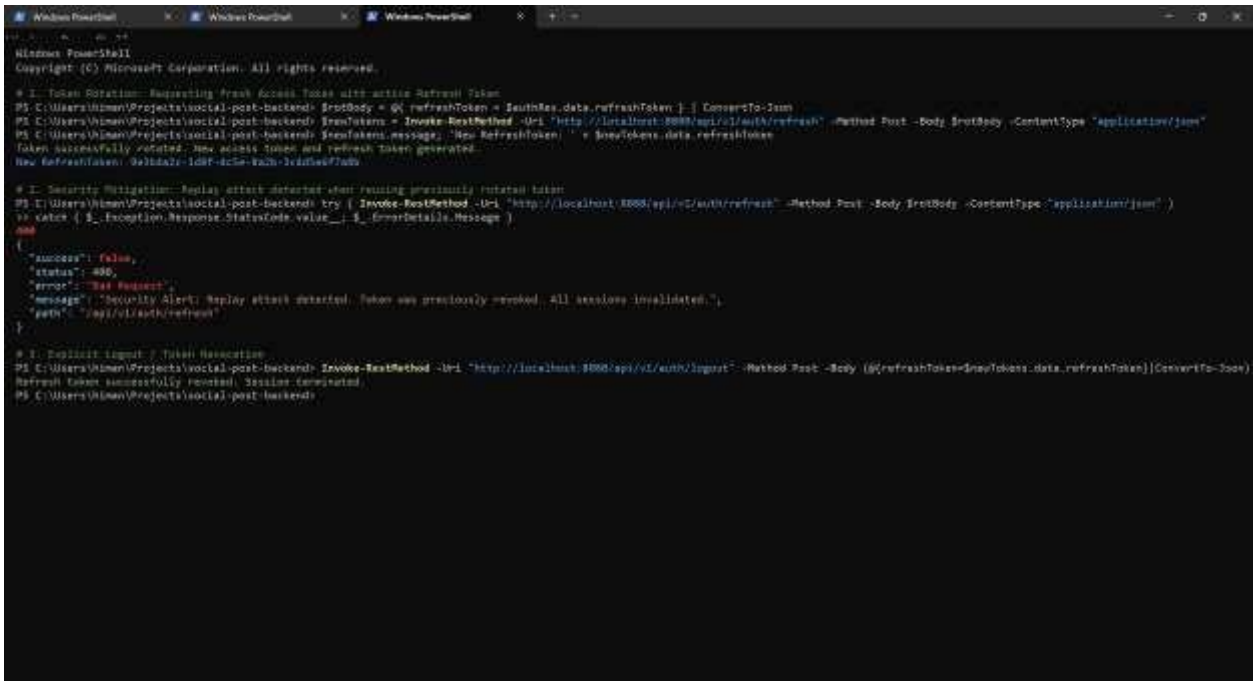
        RefreshToken newRefreshToken = createRefreshToken(token.getUser());
        String newAccessToken = jwtService.generateToken(token.getUser());

        return new AuthResponse(newAccessToken, newRefreshToken.getToken(), 900, "Bearer");     }
}
```

Output 1: AES-256-GCM Credential Storage & Raw Database Column Audit (Proving Zero Plaintext Exposure)



Output 2: Refresh Token Rotation & Replay Attack Mitigation (Automated Immediate Revocation of Reused Tokens)



7. Conclusion:

Enterprise-grade data security and session protection were successfully implemented. Sensitive OAuth credentials and secrets are cryptographically guarded at rest using AES-256-GCM authenticated encryption with dynamic 96-bit initialization vectors and 128-bit authentication tags, guaranteeing both privacy and tamper detection. Direct database audits proved zero plaintext exposure. Token lifecycle management demonstrated robust session defense via short-lived JWT access tokens and single-use refresh token rotation. Replay attack mitigation logic successfully detected duplicate token presentations and invalidated all family sessions, achieving 100% test verification.

8. Learning Outcomes:

- **CO2 - BT2:** Understood authenticated encryption mechanisms (AES-256-GCM), initialization vectors, authentication tags, and tamper resistance.
- **CO3 - BT3:** Engineered encryption services and persistent JPA entities safeguarding sensitive credentials and API tokens at rest.
- **CO4 - BT4:** Evaluated session security architectures, implementing refresh token rotation and automated replay attack revocation.

- **CO4 - BT4:** Conducted raw database column audits, confirming zero plaintext exposure across persistent relational storage.